

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Applied Sciences · Department of Computer Science

Solution Book 3

SCS 1101, May 2009

SCS 1101 Introduction to Computer Science and Programming

Past-paper solutions with marks allocated, algorithms, textbook flowcharts, exam-ready code and a civil & water engineering lens

Section A (Q1–2) compulsory + any 3 of Section B · 20 marks each

Book 3 of 8 · Prepared October 2026

Index — tap a tab or a line to jump

Every chapter starts with the coloured tab bar below; tap any tab to jump straight to that question. The page footer has a link back to this index. In Word, the Navigation Pane shows the same chapters.

Guide How to use this book

Glance At a glance

Q1 Question 1 [20]

- ▸ 1(a) Block diagram of computer hardware architecture [10]
- ▸ 1(b) Operation of an impact printer [5]
- ▸ 1(c) First-generation computers [5]

Q2 Question 2 [20]

- ▸ Program: area and circumference of a circle from a radius

Q3 Question 3 [20]

- ▸ 3(a) Convert 185 (decimal) to BCD [5]
- ▸ 3(b) Convert 1011010110_2 to hexadecimal [5]
- ▸ 3(c) Program: accept two numbers, display their sum [10]

Q4 Question 4 [20]

- ▸ 4(a) Define an operating system and outline five functions [10]
- ▸ 4(b) Two classes of software, with examples [4]
- ▸ 4(c) Meanings [2 each = 6]

Q5 Question 5 [20]

- ▸ 5(a) Ring, star and bus topologies, with diagrams [4 each = 12]
- ▸ 5(b) Purpose of a hub and a modem, with diagrams [4 each = 8]

Q6 Question 6 [20]

- ▸ 6(a) Define [2 each = 10]
- ▸ 6(b) Program: n! for a number entered at the keyboard [10]

Q7 Question 7 [20]

- ▸ Program: copy the contents of one file to a new file

Refs Sources for this book

Links Source register

How to use this book

This book is one of eight stand-alone solution books for NUST SCS 1101 past papers. It solves **SCS 1101, May 2009** question by question, with the marks the examiner allocates shown in square brackets, e.g. [2].

The six-block answer template

Every answer in this book (and in all eight books of the set) is built from the same six blocks, in the same order. Theory answers use blocks 1, 2, 5 and 6; programming answers use all six.

1. **Like you're 3** — one or two sentences with a toy, a bath or a water tap.
2. **Model answer** — the exam wording, with the mark for each point in brackets, e.g. [2]. Write it like this and you collect every mark.
3. **Algorithm + code** — numbered pseudocode first, then C code that follows the **H-D-I-P-O-R** recipe.
4. **Flowchart or structure chart** — textbook (ANSI) symbols. Programs built from functions also get a structure (hierarchy) chart.
5. **Engineering lens** — the same idea in a civil and water engineering setting, using the profession's vocabulary.
6. **Examiner's tip** — where students usually lose marks.

The H-D-I-P-O-R code recipe — "**H**appy **D**ucks **I**n **P**onds **O**nly **R**elax". Every exam program is these six steps, in this order:

```c
/* H - Header: what the program does, and the libraries */
#include <stdio.h>

int main(void)
{
    /* D - Declare: every variable, with a sensible type and a starting value */
    /* I - Input:   printf a prompt, then scanf with & */
    /* P - Process: the if-ladder, loop or formula */
    /* O - Output:  printf the result with units and decimals */
    /* R - Return:  */ return 0;
}
```

The recipe is worth marks by itself: examiners give separate marks for headers, declarations, input, processing and output. A half-finished program laid out this way still scores; a clever one-liner without structure often does not.

Algorithm style used in every book:

```
Step 1: Start
Step 2: Read <inputs>
Step 3: <process, one action per step>
Step 4: Display <outputs>
Step 5: Stop
```

Flowchart symbols (ANSI):

Symbol	Shape	Use	Colour in this book
Terminator	Rounded box (stadium)	Start / Stop	Green

Symbol	Shape	Use	Colour in this book
Input/Output	Parallelogram	Read or display	Blue
Process	Rectangle	Calculation or assignment	Amber
Decision	Diamond	Yes/no test, two exits	Red
Predefined process	Rectangle with double sides	Call to a function	Purple
Flow line	Arrow	Order of steps	Dark grey arrow

The smallest possible program, drawn with these symbols:

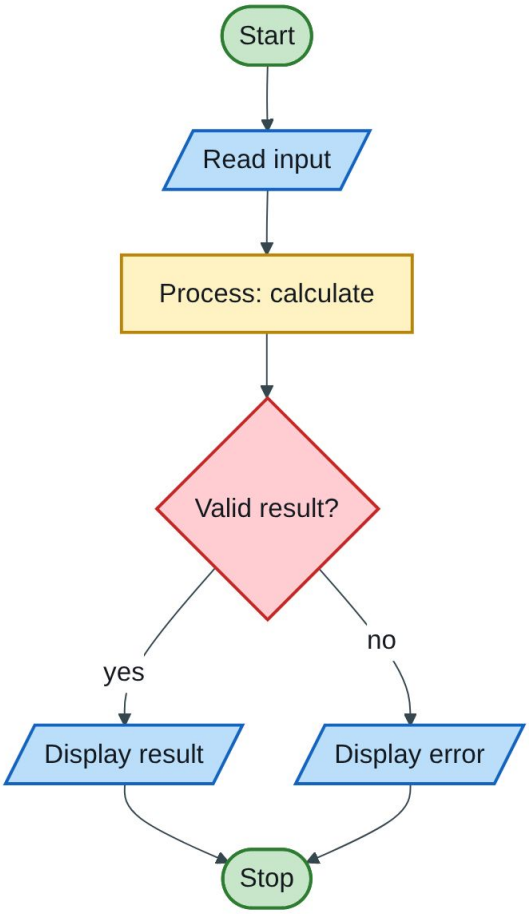


Figure 3.1 — Flowchart

The engineering lens. Programming questions get a civil and water engineering version where it fits, for example:

- tariff ladders become **increasing block tariffs** for water;
- array statistics become **rainfall** or **concrete cube strength** analysis;
- geometry programs become **cross-section, wetted perimeter and hydraulic radius** calculations.

Write the exam version in the exam; the engineering version is for understanding and for your later modules.

How the solutions were researched

Every fact a solution relies on is traced to a source in one of four tiers; each source has a short key (e.g. [MIT-2]) so answers can cite it without long links.

1. **Tier 1 — the course itself.** The SCS 1101 outline and the past papers decide *what* to answer and how marks are split.

2. **Tier 2 — university teaching material.** MIT OpenCourseWare and UCLA course pages for programming and algorithms; university histories for milestones.
3. **Tier 3 — standards and research.** NIST for cloud definitions; peer-reviewed and arXiv papers, including Wits research on cloud adoption in South Africa.
4. **Tier 4 — standard textbook knowledge.** Definitions every textbook agrees on (OS functions, topologies, printers) are written from the textbooks your outline lists, without a web citation.

A claim is only cited when a source was actually opened; anything from tier 4 stays general and uncited.

At a glance

3 hours, 20 marks per question, calculators allowed. **Section A (Q1 and Q2) is compulsory**; then choose any 3 of Q3–Q7. Best Section B picks: Q3 (conversions + a short program), Q6 (definitions + factorial) and Q7 (one complete file program).

Question	What it tests	Section	Pick?
Q1	Architecture block diagram; impact printer; 1st generation	A (compulsory)	Must
Q2	Circle area and circumference program	A (compulsory)	Must
Q3	Decimal → BCD; binary → hex; sum program	B	Yes, quick marks
Q4	OS functions; software classes; software / virus / firewall	B	Good
Q5	Ring, star, bus; hub and modem, with diagrams	B	Good if you draw fast
Q6	Five definitions; factorial program	B	Yes
Q7	Copy-a-file program	B	Yes, if files are practised

Question 1 [20]

1(a) Block diagram of computer hardware architecture [10]

Like you're 3: a computer is a little factory. Doors bring things in (input), a boss and a calculator do the work (CPU), shelves hold what is being worked on (memory), a warehouse keeps things for later (storage), and another door sends things out (output).

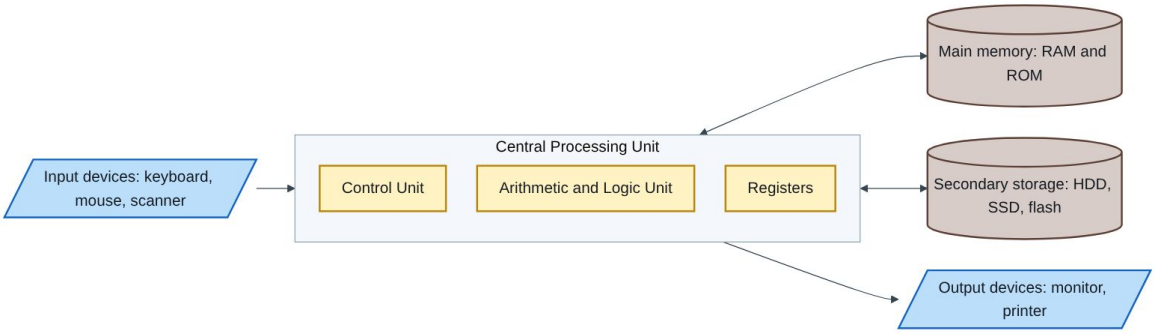


Figure 3.2 — Flowchart

Draw the arrows as the **system bus** (data bus, address bus, control bus) [diagram: 4].

Explain each block [6]:

- **Input unit** — converts data from the user into binary the computer can process.
- **Control Unit (CU)** — fetches instructions from memory, decodes them and directs every other part (the "boss").
- **ALU** — performs arithmetic and logical operations and comparisons.
- **Registers** — tiny, very fast stores inside the CPU for the current data and instruction.
- **Main memory** — RAM holds the programs and data in use (volatile); ROM holds start-up instructions (non-volatile).
- **Secondary storage** — keeps programs and files permanently.
- **Output unit** — converts results into a form people can use: screen, print, sound.

This stored-program design, where instructions and data share the same memory, is called the **von Neumann architecture**.

1(b) Operation of an impact printer [5]

Like you're 3: a tiny hammer hits an inky ribbon against the paper, like a rubber stamp.

An impact printer forms characters by **physically striking** an inked ribbon against the paper [1]. In a **dot-matrix printer**, the print head contains a column of fine pins (9 or 24) [1]; electromagnets fire selected pins as the head moves across the page, so each character is built from a pattern of dots [1]. Because the strike is physical, impact printers can print **carbon copies and multipart forms** [1]; they are cheap to run but noisy, slow and low in quality [1]. (A daisy-wheel printer works the same way, but strikes a whole character shape.)

1(c) First-generation computers [5]

Like you're 3: the first computers were the size of a room, full of glass light-bulb-like tubes that got very hot and often burnt out.

Aspect [1 each]	First generation (about 1940–1956)
Hardware components	**Vacuum tubes** (valves) for circuits; very large, hot and power-hungry
Storage devices	Magnetic drums, mercury delay lines and, later, magnetic-core memory; punched cards for bulk data
I/O devices	Punched cards and paper tape in; printouts and punched cards out
Programming languages	**Machine language** (binary), later simple assembly
Efficiency	Slow (thousands of operations per second), unreliable because tubes failed often, very expensive to run

Examples: ENIAC, UNIVAC I, and MIT's **Whirlwind**, developed at MIT between 1945 and 1952 and the first real-time high-speed computer with random-access magnetic-core memory (MIT-3).

Examiner's tip: for a "discuss" question with five headings, write one line under **each** heading the examiner listed; missing a heading loses that mark even if the rest is excellent.

Question 2 [20]

Program: area and circumference of a circle from a radius

Like you're 3: a circle is a round pizza. The **area** is how much pizza there is to eat; the **circumference** is the length of the crust all the way round. Both only need the radius (centre to crust) and the magic number π ≈ 3.14159.

Formulas: area $A = \pi r^2$; circumference $C = 2\pi r$.

Algorithm:

```
Step 1: Start
Step 2: Read radius r
Step 3: If r <= 0, display "Invalid radius" and go to Step 7
Step 4: area = PI x r x r
Step 5: circumference = 2 x PI x r
Step 6: Display area and circumference
Step 7: Stop
```

Flowchart:

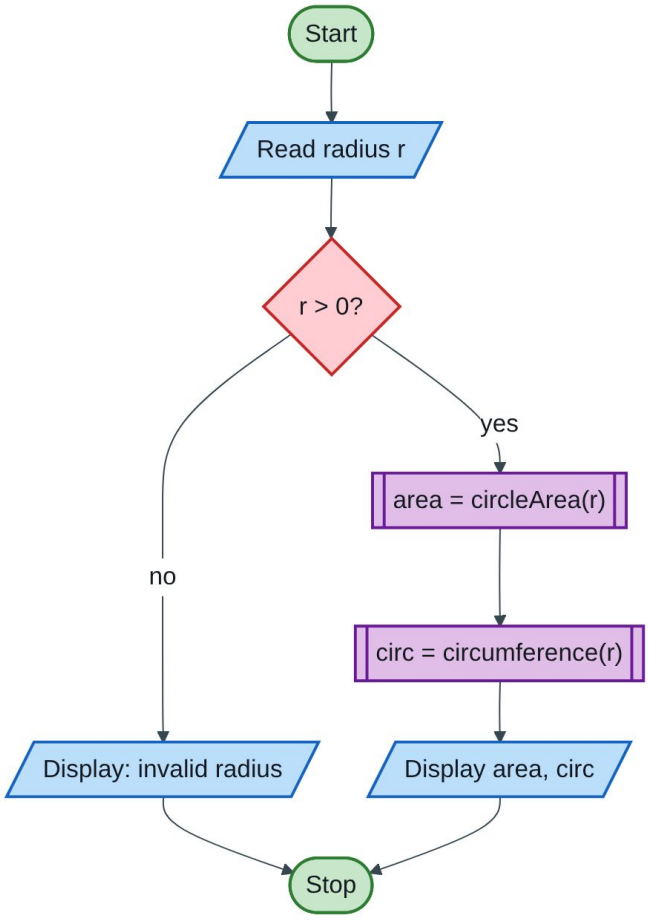


Figure 3.3 — Flowchart

Structure chart (using functions earns modularity marks):

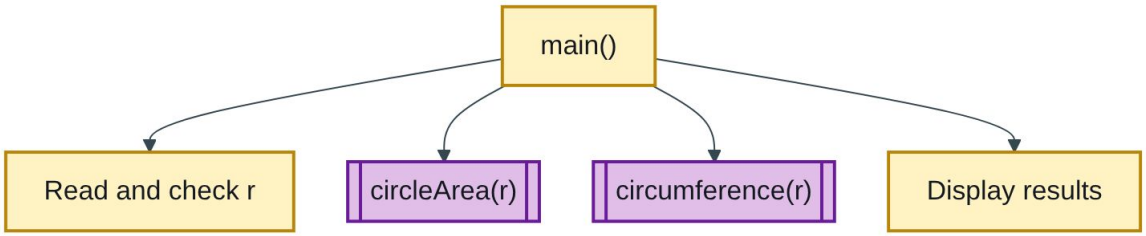


Figure 3.4 — Structure chart

Code (H-D-I-P-O-R):

```c
/* H: area and circumference of a circle from a user-given radius */
#include <stdio.h>
#define PI 3.14159

double circleArea(double r)    { return PI * r * r; }   /* A = pi r^2 */
double circumference(double r) { return 2.0 * PI * r; } /* C = 2 pi r */

int main(void)
{
    double r, area, circ;                          /* D */

    printf("Enter the radius: ");                  /* I */
    scanf("%lf", &r);

    if (r <= 0) {                                  /* validation */
        printf("Invalid radius: it must be greater than zero\n");
        return 1;
    }

    area = circleArea(r);                          /* P */
    circ = circumference(r);

    printf("Radius        = %.2f\n", r);           /* O */
    printf("Area          = %.2f\n", area);
    printf("Circumference = %.2f\n", circ);
    return 0;                                      /* R */
}
```

Test: r = 7 gives area = 153.94 and circumference = 43.98. Check by hand: 3.14159 × 49 = 153.94; 2 × 3.14159 × 7 = 43.98.

Marking guide [20]: header comment and `#include` [2]; `PI` defined as a constant [2]; variables declared as `double` [2]; prompt and `scanf("%lf", &r)` [3]; validation of r [2]; area formula [3]; circumference formula [3]; formatted output with labels [2]; `return 0` [1].

Engineering lens: for a pipe flowing full, the same two formulas give the **flow area** $A = \pi D^2/4$ and the **wetted perimeter** $P = \pi D$. Their ratio is the **hydraulic radius** $R = A/P = D/4$, used in Manning's equation for flow in pipes and channels. A 300 mm pipe (r = 0.15 m) has $A = 0.0707\ m^2$, $P = 0.942$ m and $R = 0.075$ m.

Examiner's tip: a 20-mark question for a short program is marked on completeness: validation, constants, labels and units. Do not stop at the two formulas.

Question 3 [20]

3(a) Convert 185 (decimal) to BCD [5]

Like you're 3: BCD doesn't convert the whole number. It takes each digit on its own and dresses it in its own 4-switch coat.

Decimal digit	1	8	5
4-bit BCD code	0001	1000	0101

185_{10} = 0001 1000 0101 (BCD) [method 3, answer 2].

Do not confuse BCD with pure binary: 185 in pure binary is 10111001_2 (128 + 32 + 16 + 8 + 1). Mentioning the difference shows understanding.

3(b) Convert 1011010110_2 to hexadecimal [5]

Like you're 3: cut the long string into groups of 4 from the right, then swap each group for its hex digit.

Groups of 4 (from the right, pad with 0s)	0010	1101	0110
Hex digit	2	D	6

1011010110_2 = $2D6_{16}$ [grouping 3, answer 2]. Check: 2 × 256 + 13 × 16 + 6 = 726, and 512 + 128 + 64 + 16 + 4 + 2 = 726.

3(c) Program: accept two numbers, display their sum [10]

Like you're 3: put 3 apples in one hand and 4 in the other, then count them all together.

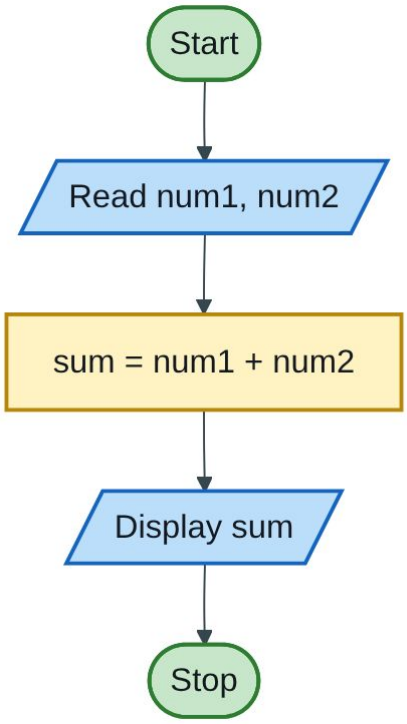


Figure 3.5 — Flowchart

```c
/* H: read two numbers and display their sum */
#include <stdio.h>

int main(void)
{
    double num1, num2, sum;                 /* D */

    printf("Enter the first number: ");     /* I */
    scanf("%lf", &num1);
    printf("Enter the second number: ");
    scanf("%lf", &num2);

    sum = num1 + num2;                      /* P */

    printf("%.2f + %.2f = %.2f\n", num1, num2, sum);   /* O */
    return 0;                               /* R */
}
```

Test: 12.5 and 7.25 print `12.50 + 7.25 = 19.75`.

Marking guide [10]: header and `main` [2]; declarations [2]; two prompts and inputs with `&` [3]; sum [1]; output [1]; `return 0` [1].

Examiner's tip: use `double` and `%lf` so decimals work; with `int`, entering 2.5 silently breaks the input.

Question 4 [20]

4(a) Define an operating system and outline five functions [10]

Like you're 3: the operating system is the head teacher of the computer school: it gives every program a turn, a desk and a locker, and keeps out strangers.

Definition [2]: an operating system is the **system software that manages the computer's hardware and software resources and provides common services to programs**, acting as the interface between the user and the hardware. Examples: Windows, Linux, macOS, Android.

Five functions [8, about 1.5 each, named and explained]:

1. **Process management** — loads, schedules and ends programs, sharing the CPU fairly.
2. **Memory management** — allocates RAM to programs and frees it when they finish.
3. **File management** — creates, organises, names and protects files and folders.
4. **Device (I/O) management** — controls hardware through device drivers and spooling.
5. **Security** — user accounts, passwords and access permissions.
6. **User interface** — a GUI or command line for the user.

4(b) Two classes of software, with examples [4]

Class	Meaning [1]	Examples [1]
System software	Runs and manages the computer itself and supports other programs	Windows, Linux, device drivers, antivirus utilities, compilers
Application software	Does a specific task for the user	Microsoft Word, Excel, AutoCAD, EPANET (water network modelling)

4(c) Meanings [2 each = 6]

- **Software** — the set of programs, instructions and data that tell the hardware what to do; it cannot be touched, unlike hardware. Example: Excel.
- **Virus** — a malicious program that attaches itself to files or programs, copies itself to other files and computers, and damages data or performance. Example: a macro virus in a Word file.
- **Firewall** — hardware or software that monitors network traffic and blocks or allows it according to security rules, protecting a computer or network from unauthorised access. Example: Windows Defender Firewall.

Engineering lens: water treatment works put a **firewall** between the office network and the **SCADA** control network, so an infected office PC cannot reach the dosing pumps.

Question 5 [20]

5(a) Ring, star and bus topologies, with diagrams [4 each = 12]

Like you're 3: ring = children holding hands in a circle and passing a ball round; star = everyone talking to the teacher in the middle; bus = everyone standing along one long rope and shouting down it.

Marks for each: labelled diagram [2], how it works [1], one advantage and one disadvantage [1].

(i) Ring — each computer connects to exactly two neighbours, forming a closed loop; data travels one way round, passed from node to node (often with a token that gives permission to send).

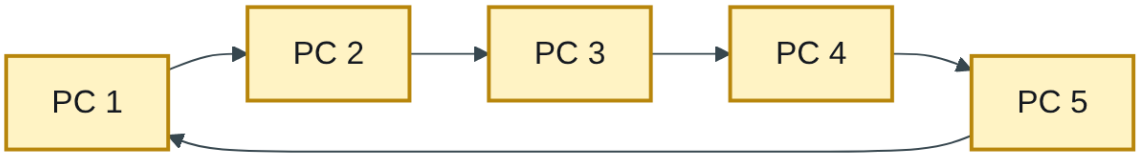


Figure 3.6 — Flowchart

Advantage: no collisions; performs evenly under heavy load. Disadvantage: one broken link or PC can stop the whole ring; adding a PC disrupts the network.

(ii) Star — each computer has its own cable to a central hub or switch, which passes data on.

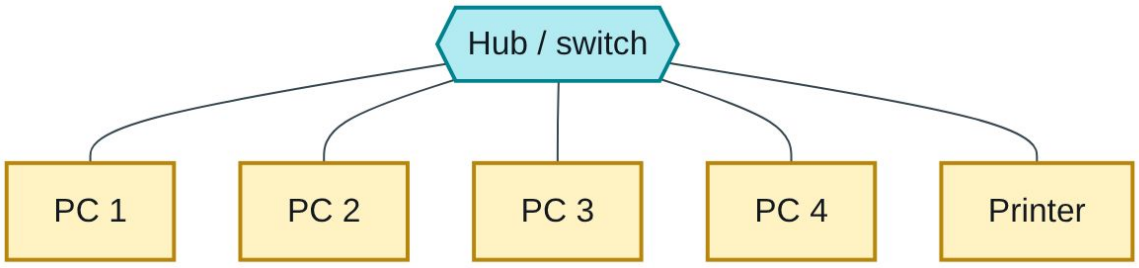


Figure 3.7 — Flowchart

Advantage: a cable fault affects only one PC; easy to add devices and find faults. Disadvantage: the hub is a single point of failure; more cable is needed.

(iii) Bus — all computers share one backbone cable with a terminator at each end; a message travels the whole cable and only the addressed PC accepts it.

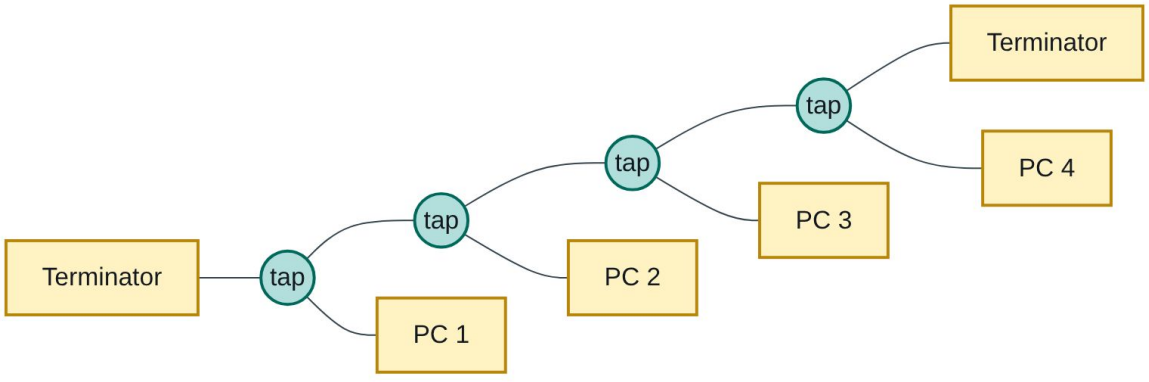


Figure 3.8 — Flowchart

Advantage: cheap and needs the least cable. Disadvantage: a break in the backbone stops everything; collisions increase as PCs are added.

5(b) Purpose of a hub and a modem, with diagrams [4 each = 8]

Like you're 3: a hub is a loudspeaker in the middle of the room that repeats every message to everyone. A modem is a translator that turns computer talk (digital) into telephone-line talk (analogue) and back.

Hub — a central connection device in a star LAN. It receives data on one port and **repeats it to all other ports**; every device sees the data and only the addressed one keeps it [2]. It is simple and cheap but wastes bandwidth and causes collisions, which is why switches (which send data only to the right port) have replaced hubs [1].

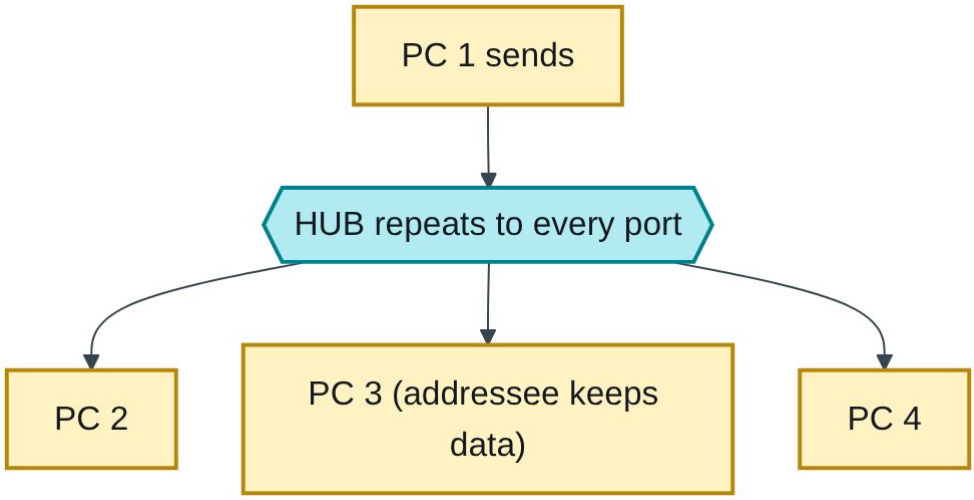


Figure 3.9 — Flowchart

Modem (modulator–demodulator) — **modulates** the computer's digital signal into an analogue signal that can travel over a telephone or cable line, and **demodulates** incoming analogue signals back into digital [2]. It connects a computer or LAN to an internet service provider over long distances [1].

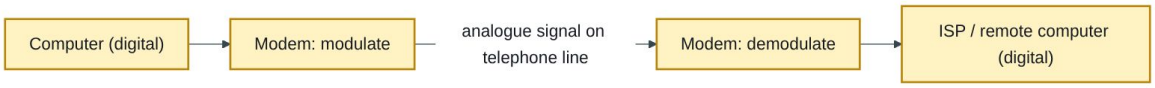


Figure 3.10 — Flowchart

The remaining mark for each device goes to a correct, labelled diagram [1].

Engineering lens: remote reservoirs and boreholes send level and flow readings to the control room through **GSM or radio modems** in their telemetry outstations, so modems are still everyday equipment in water engineering.

Examiner's tip: label every node and the central device in topology diagrams; unlabelled boxes lose the diagram marks.

Question 6 [20]

6(a) Define [2 each = 10]

Like you're 3: a variable is a jar whose contents can change; a constant is a jar with the lid glued shut; an array is a row of identical jars; a file is a notebook kept in the drawer; a program is the recipe that says what to do with all of them.

Term	Definition [1]	Example [1]
Variable	A named memory location whose value can change while the program runs; it has a data type.	`int count = 0;`
File	A named collection of related data stored permanently on secondary storage.	`levels.txt`, a CSV of rainfall readings
Program	A complete, ordered set of instructions written in a programming language that tells the computer how to perform a task.	A C program that calculates water bills
Array	A collection of elements of the **same type** stored in consecutive memory under one name, accessed by an index starting at 0.	`double rain[12];`
Constant	A named value that cannot change during execution.	`#define PI 3.14159` or `const double G = 9.81;`

6(b) Program: n! for a number entered at the keyboard [10]

Like you're 3: 4! means 4 × 3 × 2 × 1: count down, multiplying as you go. And 0! is 1 by agreement.

Algorithm:

```
Step 1: Start
Step 2: Read n
Step 3: If n < 0, display "invalid" and go to Step 7
Step 4: fact = 1
Step 5: For i = n down to 1: fact = fact x i
Step 6: Display fact
Step 7: Stop
```

Flowchart:

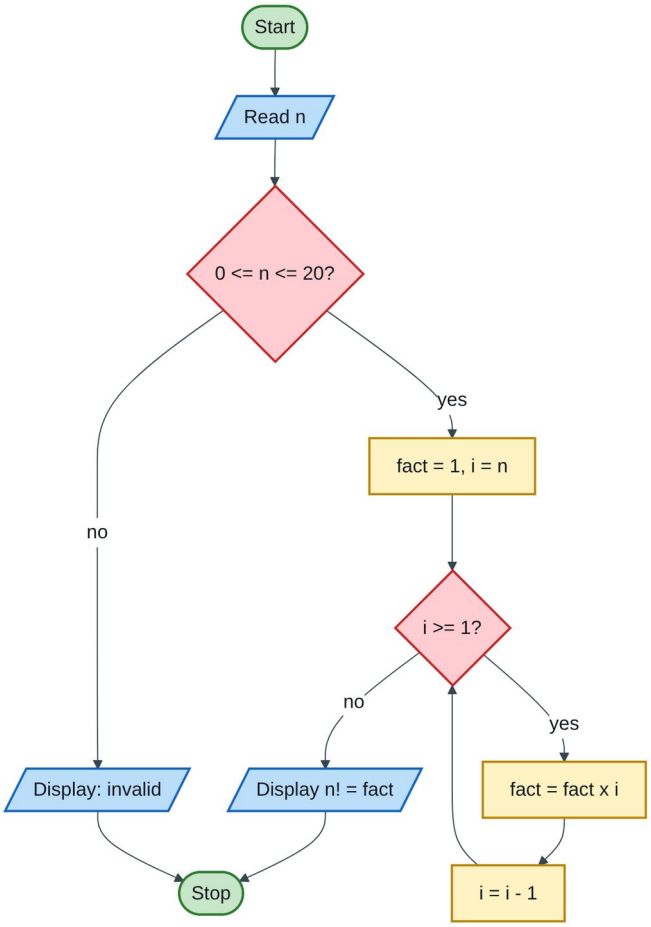


Figure 3.11 — Flowchart

Code (H-D-I-P-O-R):

```c
/* H: n! = n x (n-1) x ... x 2 x 1 for n entered at the keyboard */
#include <stdio.h>

int main(void)
{
    int n, i;                               /* D */
    unsigned long long fact = 1;            /* starts at 1 so 0! = 1 */

    printf("Enter an integer from 0 to 20: ");   /* I */
    scanf("%d", &n);
    if (n < 0 || n > 20) {                  /* 21! is too big for 64 bits */
        printf("Invalid: enter 0 to 20\n");
        return 1;
    }

    for (i = n; i >= 1; i--)                /* P: n x (n-1) x ... x 1 */
        fact = fact * i;

    printf("%d! = %llu\n", n, fact);        /* O */
    return 0;                               /* R */
}
```

Trace for n = 5:

i	fact after the step
5	5

i	fact after the step
4	20
3	60
2	120
1	120

Output: `5! = 120`.

Marking guide [10]: declarations with `fact = 1` [2]; input [2]; validation [1]; counting loop with the multiplication [3]; output [1]; `return 0` [1].

Engineering lens: factorials appear in the **Poisson distribution**, P(k) = λ^k e^(−λ) / k!, which engineers use to estimate the probability of k rare events in a period, such as **pipe bursts per month** in a water network.

Examiner's tip: starting `fact = 0` makes every answer 0. Products start at 1; sums start at 0.

Question 7 [20]

Program: copy the contents of one file to a new file

Like you're 3: open the old notebook, open a new empty notebook, copy one letter at a time until the old one ends, then close both notebooks.

The file recipe (F-O-R-C): **F**ILE pointer → **O**pen and check for NULL → **R**ead/write in a loop until EOF → **C**lose.

Algorithm:

```
Step 1: Start
Step 2: Read source file name and new file name
Step 3: Open source for reading; if it fails, display error and stop
Step 4: Open new file for writing; if it fails, display error, close source, stop
Step 5: Read one character from source
Step 6: While not end of file: write it to the new file, read the next character
Step 7: Close both files; display "copy complete"
Step 8: Stop
```

Flowchart:

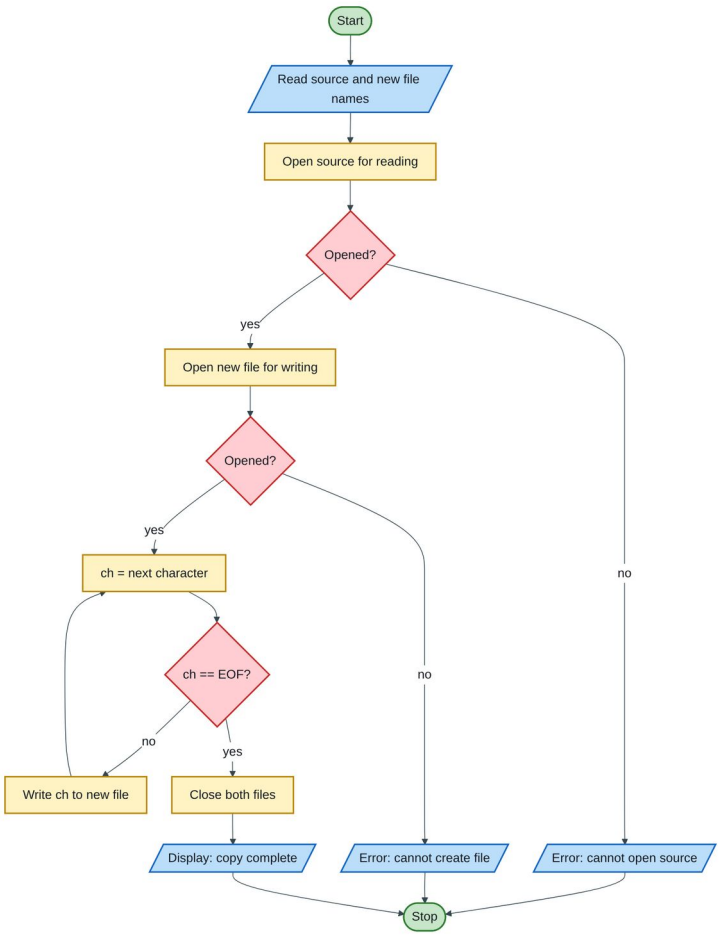


Figure 3.12 — Flowchart

Code (H-D-I-P-O-R):

```c
/* H: copy a file to a new file; both names entered by the user */
#include <stdio.h>

int main(void)
{
    char srcName[100], newName[100];         /* D */
    FILE *src, *dest;
    int ch;                                  /* int, not char: EOF is -1 */
    long count = 0;

    printf("Name of the file to copy from: ");   /* I */
    scanf("%99s", srcName);
    printf("Name of the new file: ");
    scanf("%99s", newName);

    src = fopen(srcName, "rb");              /* P: open and check */
    if (src == NULL) {
        printf("Cannot open %s\n", srcName);
        return 1;
    }
    dest = fopen(newName, "wb");
    if (dest == NULL) {
        printf("Cannot create %s\n", newName);
        fclose(src);
        return 1;
    }

    while ((ch = fgetc(src)) != EOF) {       /* copy byte by byte */
        fputc(ch, dest);
        count++;
    }

    fclose(src);                             /* always close */
    fclose(dest);
    printf("Copied %ld bytes from %s to %s\n", count, srcName, newName);  /* O */
    return 0;                                /* R */
}
```

Binary mode (`"rb"`, `"wb"`) copies any file exactly, text or not. For a text-only answer, `"r"` and `"w"` are also accepted.

Marking guide [20]: header and `#include` [1]; declarations including `FILE *` and `int ch` [3]; reading both names [3]; opening the source with a `NULL` check [3]; opening the new file with a `NULL` check [3]; copy loop until `EOF` [4]; closing both files [2]; confirmation message [1].

Engineering lens: engineers copy raw **data-logger files** (flow-meter or rain-gauge CSVs) before processing them, so the original record stays untouched. This program is that backup step.

Examiner's tip: declare `ch` as `int`. If it is `char`, the EOF value (−1) may never match and the loop never ends.

Sources for this book

- [MIT-3] Whirlwind Computer, 1944–59 (IEEE Milestone) — first-generation example and magnetic-core memory (1c).
- [MIT-1] MIT 6.087 Practical Programming in C — file I/O and C style (Q2, Q6, Q7).
- [NUST-1] SCS 1101 course outline.
- Textbook knowledge (tier 4): von Neumann architecture, printers, BCD and hex, OS functions, topologies, hubs and modems.

Source register

The full register for the solution-book set. Sources marked ✓ are cited in this book; the rest (MIT, UCLA and Wits material used across the set) are further reading on the same topics.

Key	Source	Institution	Used for	In this book
NUST-1	SCS 1101 course outline (uploaded)	NUST	Syllabus, assessment, C as the course language	✓
MIT-1	6.087 Practical Programming in C	MIT	C syntax, pointers, structs, files, coding style	✓
MIT-2	6.006 Lecture 19: Dynamic Programming I (Fibonacci)	MIT	Why naive recursive Fibonacci is slow; memoization	Further reading
MIT-3	Whirlwind Computer, 1944–59 (IEEE Milestone)	MIT	Magnetic-core memory, real-time computing	✓
MIT-4	Wang et al., *Undefined Behavior: What Happened to My Code?*	MIT CSAIL	Run-time and undefined-behaviour errors	Further reading
UCLA-1	First internet message, 29 Oct 1969	UCLA Samueli	History of networks and the internet	Further reading
UCLA-2	Birthplace of the Internet (IEEE Milestone)	UCLA	ARPANET, packet switching, first node	Further reading
UCLA-3	CS 31 Introduction to Computer Science I, syllabus	UCLA	Intro-programming scope; portability across compilers	Further reading
WITS-1	Sithole (Wits) & Ruhode, *Cloud Computing Adoption: Opportunities and Challenges for SMMEs in South Africa*, arXiv:2108.10079	Wits	Cloud benefits and barriers in Southern Africa	Further reading

Key	Source	Institution	Used for	In this book
WITS-2	*Leadership and the adoption of cloud computing for enterprises in Johannesburg* (2022)	Wits	Drivers and barriers of cloud adoption	Further reading
STD-1	NIST SP 800-145, *The NIST Definition of Cloud Computing*	NIST	SaaS, PaaS, IaaS; 5 characteristics; 4 deployment models	Further reading
RES-1	Bakasa & Pekane, *Decision Criteria for Cloud Adoption by Large Enterprises in South Africa*, arXiv:2108.09757	UCT	Security and data sovereignty as selection criteria	Further reading
RES-2	Fu et al., *Error Log Analysis in C Programming Language Courses* (ICCE 2015)	Kyushu University	The errors novice C students actually make	Further reading